Software Requirements Specification (SRS)

Intelligent Assignment Similarity and Derivation Analysis System (IAS-DAS)

Team #6

CSC4351 — Capstone 1, Section: Wednesday

Denis Joseph Jr., Mikhail Tsegaye, Tauha Hysaw, Taizo Rashid, and Vince Paul

October 2026

Purpose

The main purpose of the Intelligent Assignment Similarity and Derivation Analysis System, otherwise known as IAS-DAS, is to detect similarities in a student's programming assignment or written report that could have resulted from paraphrasing, reorganizing the text or logic structure, or even AI-assisted generation. IAS-DAS flags these assignments using multiple techniques that focus on the structural semantics of the submitted work. If IAS-DAS determines that a submitted assignment was likely not produced by the student who submitted it, it lets the instructor decide how to proceed, since academic integrity may have been compromised.

Team

Team #6 — CSC4351 Capstone 1, Section: Wednesday
Members: Denis Joseph Jr., Mikhail Tsegaye, Tauha Hysaw, Taizo Rashid, and Vince Paul

Market Comparisons

As far as plagiarism detection tools go in the academic field, Turnitin is arguably the titan of the field. Turnitin is partnered with over 20,000 institutions and has a repository containing over 1.8 billion student submissions.

Turnitin is an instructor-facing tool that is not student friendly. An individual cannot make an account on their website and use it individually; institutions purchase licenses instead. Self-plagiarism from a student's own previous submissions can also be a significant problem.

There are other tools such as Scribbr, which uses an Onyx text-matching engine and is good for students checking their own work. iThenticate (owned by Turnitin's parent company, Advance Publications) is geared toward academic researchers and journalists. Copyleaks and GPT Zero are AI-based checkers. Grammarly is easy to use and mainly focused on grammar, but it has a built-in plagiarism detector.

Overall, Turnitin dominates the market, but it could do more to be student friendly and to handle self-plagiarism better.

IAS-DAS aims to improve on this by offering both students and instructors access to the platform. With careful database management, we will track self-plagiarism and help prevent unintentional similarities.

Sources: https://hackernoon.com/are-plagiarism-detection-tools-actually-accurate ; https://freeacademictools.com/blog/free-turnitin-alternative-students-2026

Overview

Overall Description

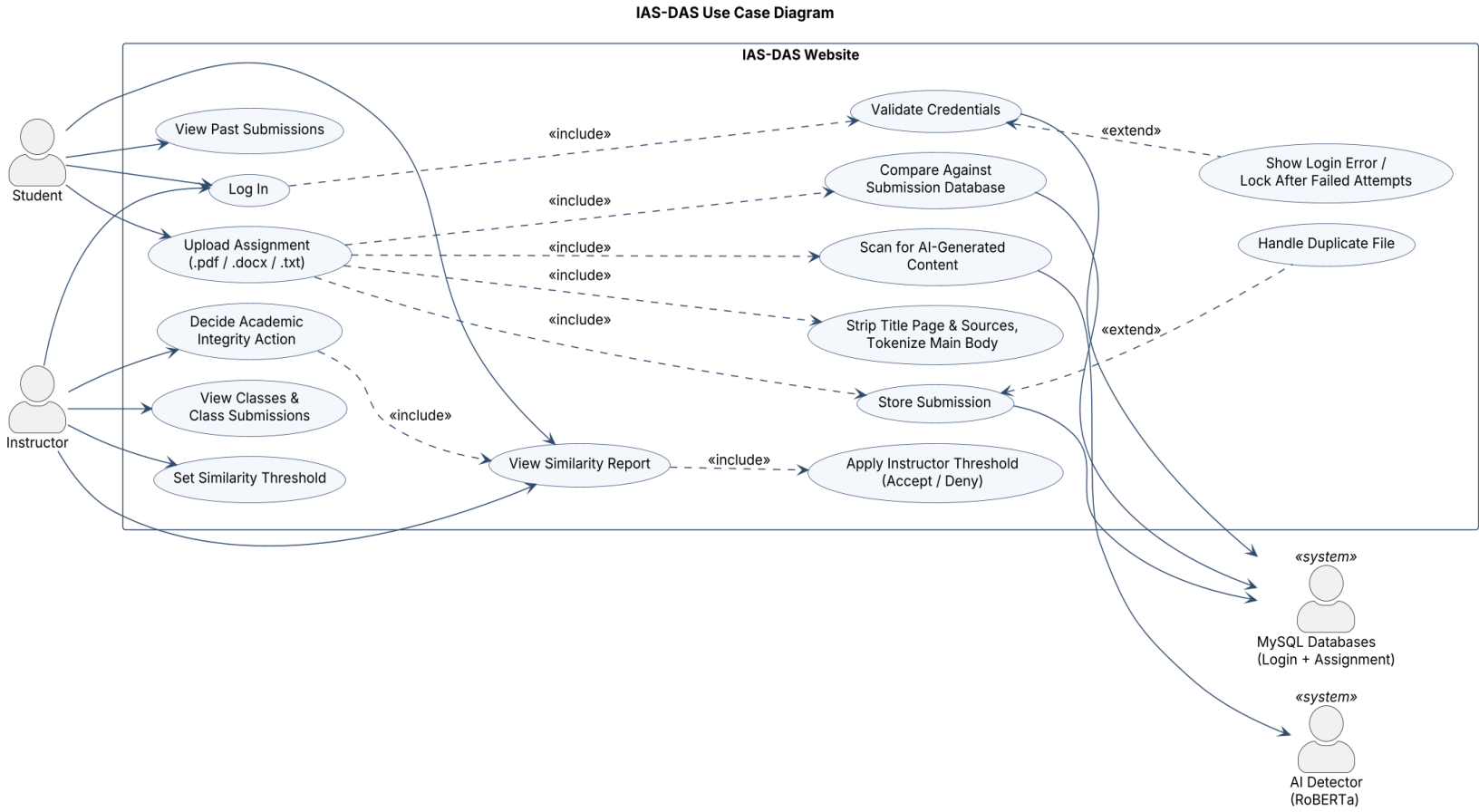
- Product Functions
    - Our product is a website that starts by accepting a file (.docx, .pdf, or .txt) as input. The uploaded file is matched against files collected in our student submission database: the title page and sources section are discarded, and the main body of text is tokenized and then compared against the database. The comparison produces a similarity percentage along with matched sources. Depending on the instructor's percentage threshold, the assignment is then accepted or denied, and the student's assignment is added to our database.
- Operating Environment
    - A stable internet connection is required, as the application depends on a network environment. The website will work best in a browser on a desktop computer. Mobile browser support is not guaranteed because of potential formatting issues.
- Design and Implementation Constraints
    - The primary constraint is the AI model used to compare a student submission against the database of prior student submissions. FERPA prevents us from building a large database of student assignments to train an AI model, so the database will be curated by our own team rather than sourced externally. Building an AI model capable of this task is difficult, particularly in avoiding self-plagiarism false positives and protecting student privacy. The specific model category is: Artificial Intelligence → Machine Learning → Deep Neural Network → Transformer Model → Detector.
- Assumptions and Dependencies
    - We assume that instructors and students will use the service on a computer with a stable browser and internet connection. The assumed file types for assignments are .pdf, .docx, and .txt. As for dependencies, the frontend will use React, and for AI text detection we will use RoBERTa, an open-source transformer-based model originally developed by Facebook AI (Meta).

- Functional Requirements
    - **Website Architecture —** Frontend: HTML, CSS, and React. Backend: a File Management System (writes to the Student Assignment Database), a User Login Database, and an AI Transformer Detector.
    - **User Login —** Input: Email or username, and password. Process: Checks the Login Database for the user. Output: Logs the user in, or returns an error message.
    - **User Login Database —** Description: A SQL Server database containing a unique id, email, username, and password for each user.
    - **Student/Instructor Portal —** Description: Allows access to past submissions, classes (for instructors), and the similarity assessment itself.
    - **File Management System —** Description: Handles retrieving and inserting assignments into the Assignment Database.
    - **Assignment Database —** Description: A SQL Server database containing all assignment submissions. Each entry has a unique primary ID, along with the date appended, course name, the submitting student's user account, and student name.
    - **AI Agent (RoBERTa) —** Description: Scans a user's submission for AI-generated content before it is appended to the database.
    - **Similarity Interface —** Description: Outputs the similarity results from the database and AI Agent. Input: A completed submission (selected from the portal's past submissions, or shown right after upload). Process: Pulls the submission's database match results and the AI Agent score, then compares the similarity percentage against the instructor's threshold. Output: Displays the overall similarity percentage, AI likelihood percentage, accepted/denied status, the submitted text with matching passages highlighted, and a list of matched sources with their individual percentages. Students see only their own results; instructors see results for all students in their classes.

- Non-functional Requirements
    - Error handling for user login should be robust and descriptive, returning messages specific to each case (no account found, wrong password, etc.).
    - The Student/Instructor Portal should have a clean, descriptive, and responsive UI with minimal input lag.
    - The File Management System must correctly insert files into the database with no conflicts; duplicate files are reported to the user with a clear error, though unique IDs still allow flexibility.
    - The similarity report must give a proper percentage interval with citations and highlighted passages in the text document.
    - The application must be responsive with minimal bugs and descriptive error handling throughout.
    - Database security must protect all user and assignment data; passwords in the User Login Database are stored as salted hashes (e.g., bcrypt), never in plain text.
    - All queries to the Login and Assignment databases use parameterized statements to prevent SQL injection.
    - Access is role-based: students can only view their own submissions and results, while instructors can only view submissions for the classes they teach.
    - Data is encrypted in transit (HTTPS/TLS), and sensitive data is encrypted at rest; student submissions are handled in line with FERPA and are never shared outside the platform.
    - Repeated failed login attempts temporarily lock the account, and the database is backed up regularly to prevent data loss.

- External Interface Requirements
    - User Interface: Web-based interface for all interactions, built with React.
    - External System Interfaces: Database — SQL, specifically MySQL, for the Login and Assignment databases.

- Use Case


IAS-DAS Use Case Diagram
IAS-DAS Website
Student
Instructor
View Past Submissions
Log In
Upload Assignment
(.pdf / .docx / .txt)
Decide Academic
Integrity Action
View Classes &
Class Submissions
Set Similarity Threshold
Validate Credentials
Compare Against
Submission Database
Scan for AI-Generated
Content
Strip Title Page & Sources,
Tokenize Main Body
Store Submission
View Similarity Report
Apply Instructor Threshold
(Accept / Deny)
Show Login Error /
Lock After Failed Attempts
Handle Duplicate File
«include»
«include»
«include»
«include»
«include»
«include»
«include»
«extend»
«extend»
«system»
MySQL Databases
(Login + Assignment)
«system»
AI Detector
(RoBERTa)